

LEDGER TWIN

Payment & Identity Reconciliation Agent

Updated Build Spec for Lemma × Comma Capital Hackathon
Sep 13, 2026 · Judges: Phillip Li & Akira Tong (Arga Labs)

Locked Package	A Demo Narrative · B Scorecard-as-Product · D Ambiguity Triad · E Eval Harness · G Strict Match Policy · H Killer Demo Script
External Services	Stripe (test) · Gmail API · Airtable · Slack — 4 integrations (≥3 required)
Agent Stack	Python 3.11 · FastAPI · LangGraph · Claude (Anthropic) · RapidFuzz · SQLite
Rubric Target	Technical 30% · Reliability & Eval 25% · Usefulness 20% · Originality 15% · Demo Clarity 10%

One-line pitch: Your books lie until payments, emails, and client names agree — Ledger Twin is the agent that reconciles them with idempotency guards, fuzzy identity resolution, and human-in-the-loop only when confidence is low.

Build-ready · Demo-scripted · Eval-harness included

1. Problem & Why This Wins

Freelancers and small agencies get paid through inconsistent channels: Stripe, a bank transfer only mentioned in an email, a client paying under a different name than the contact record, or a webhook that fires twice. There is no single trustworthy ledger.

Ledger Twin is that ledger — built around the three ambiguity problems Phillip Li has named as what breaks production agents:

- **Same entity?** — Are two records the same person/company?
- **Already happened?** — Has this action already been processed (idempotency)?
- **Who should act?** — Agent auto-execute vs human approval via Slack?

The idempotency-guard layer mirrors Akira Tong's fraud-detection background at Stripe. The self-built failure-injection eval harness mirrors Arga Labs' sandbox-testing methodology. Together with a live reliability scorecard, this package maximizes the 25% Reliability & Evaluation and 10% Demo Clarity slices of the rubric.

1.1 Locked Improvements (Default Package)

ID	Improvement	What We Ship
A	Demo Narrative	Frame as “books lie until payments, emails, and names agree.” One persona: freelancer/agency ops.
B	Scorecard-as-Product	Dashboard is the hero artifact: live metrics, not a side page.
D	Ambiguity Triad UI	Every event card labels Same entity? / Already happened? / Who acts? + confidence.
E	Eval Harness	`POST /eval/run` injects the full test suite and writes pass/fail into the scorecard.
G	Strict Match Policy	Never auto-close if name score < 90 OR amount not exact. Near-miss → Slack only.
H	Killer Demo Script	Wrong-name → Slack Approve → duplicate webhook blocked → scorecard ticks. Rehearse x2.

2. External Services (≥3 Required)

Four production-style integrations. Claude/Anthropic is the reasoning model inside the agent — not counted as an “external app,” but is required for entity judgment and email parsing.

Service	Role in Product	Auth	Key Calls
Stripe (Test Mode)	Payment & invoice events	Test secret + webhook signing secret	paymentIntents.list, invoices.list, webhooks: payment_intent.succeeded / invoice.paid
Gmail API	Read payment emails (“paid via e-transfer”, “sent the wire”); send receipts on high-confidence close	OAuth 2.0 — gmail.readonly + gmail.send	users.messages.list / get / send
Airtable	Canonical human-facing ledger: Clients, Invoices, Payments, EventLog	Personal Access Token	REST list/create/update records
Slack	HITL approval when confidence is low; duplicate / mismatch alerts	Bot token — chat:write + interactivity	chat.postMessage, Block Kit Approve/Reject, events callback

2.1 Credentials Checklist (Hour 0)

- Anthropic API key (Claude)
- Stripe test account + webhook signing secret
- Google Cloud project: Gmail API enabled + OAuth consent screen
- Airtable base + Personal Access Token
- Slack workspace + Slack App (chat:write + interactivity) + public HTTPS URL (Railway / ngrok)

3. Tech Stack

Layer	Choice
Backend / webhooks	Python 3.11 + FastAPI
Agent orchestration	LangGraph (visual graph for demo; native retries)
LLM	Claude via Anthropic API — entity match, email parse, verbal judgment
Fuzzy matching	RapidFuzz token-sort-ratio before LLM fallback
Idempotency store	SQLite with UNIQUE constraint on event hash
Human ledger	Airtable (Clients, Invoices, Payments, EventLog)
Queue	asyncio task queue (hackathon scale — no Redis/Celery)
Dashboard	React SPA or plain HTML + Chart.js polling `/scorecard`
Hosting	Railway or Vercel for public webhooks; ngrok as local fallback

4. Architecture & End-to-End Workflow

Every inbound signal becomes a normalized Event, then walks a single LangGraph pipeline. Nothing mutates Airtable or sends email until the Confidence Router decides auto-execute vs Slack HITL.

4.1 LangGraph Node Pipeline

1. **Ingest.** Stripe webhook, Gmail poll, or manual/eval trigger arrives at FastAPI.
2. **Normalize.** Convert to common Event `{source, type, external\_id, amount, name\_hints, raw, timestamp}`.
3. **Idempotency Guard.** Hash `(source, external\_id, amount, date)`. SQLite UNIQUE — duplicates → `blocked\_duplicates`, stop.
4. **Entity Resolver.** RapidFuzz vs Airtable Clients. ≥ 90 auto-match · 60–89 Claude judgment · < 60 new client + Slack flag.
5. **Payment↔Invoice Matcher.** Strict policy (G): exact amount required for auto-close; near-miss → Slack only.
6. **Ambiguity Triad Annotator (D).** Attach answers + confidence for Same entity? / Already happened? / Who acts?
7. **Confidence Router.** High → execute (Airtable update + optional Gmail receipt). Low → Slack Block Kit Approve/Reject.
8. **Execute / Await Human.** Apply mutation; write EventLog; update scorecard aggregates.

4.2 Strict Match Policy (G) — Rules of Engagement

- Auto-close invoice **only if** name similarity ≥ 90 **AND** payment amount == invoice amount (within \$0.01).
- Name 60–89 → Claude yes/no/uncertain; uncertain or no → Slack, never silent merge.
- Amount short/over by any material amount → Slack with computed delta; never mark fully paid.
- Duplicate hash → block; log; do not send second receipt.
- No resembling client → create Airtable client with `needs\_review=true` + Slack notify.

4.3 Data Model (Minimum)

SQLite: `idempotency\_keys(hash UNIQUE, source, external\_id, created\_at)` · `blocked\_duplicates` · optional pending_actions.

Airtable tables: Clients (name, email, aliases) · Invoices (client_id, amount, status, remaining) · Payments (client_id, invoice_id, amount, source, external_id) · EventLog (event_id, triad answers, confidence, action, outcome, human_decision).

5. Features — How Each Works

F1. Idempotency Guard

Every event hashed on (source, external transaction ID, amount, ISO date truncated to day). UNIQUE insert in SQLite; duplicate insert fails fast → routed to blocked_duplicates. Prevents double-webhook and double-receipt failure modes (Akira/Stripe-aligned).

F2. Fuzzy Entity Resolution + Claude Fallback

RapidFuzz token-sort-ratio against Airtable clients. ≥90 auto-merge. 60–89 → Claude structured prompt with both records' full context (yes/no/uncertain + reasoning). <60 → new client. Strict policy forbids silent wrong merges.

F3. Payment-to-Invoice Matching

For unmatched Stripe payments, query open invoices for resolved client. Exact amount → auto-close. Near-miss → flag partial/scope-change hypothesis and Slack. No exact amount → no auto-close.

F4. Slack Human-in-the-Loop

Low-confidence actions post Block Kit with candidate records side-by-side and Approve / Reject / Merge. FastAPI interactivity endpoint executes or discards pending action and writes human decision to EventLog.

F5. Ambiguity Triad UI (D)

Every event card and Slack message shows three labeled answers: Same entity? Already happened? Who acts? plus confidence %. This makes the judge thesis visible in the product, not just the pitch.

F6. Reliability Scorecard Dashboard (B)

React/HTML page polls `GET /scorecard`. Aggregates: entity-resolution accuracy %, duplicate-catch rate %, false-merge rate %, escalation rate %, silent-wrong-completion count (from eval), recent EventLog feed.

F7. Eval Harness `/eval/run` (E)

One button or `POST /eval/run` injects the verification suite (Section 7), runs the agent against fixtures, writes pass/fail per case into EventLog/scorecard. Demo can show “we evaluate like production.”

F8. Gmail Payment Signal + Receipt

Poll/read messages mentioning payment language; normalize as Events. On high-confidence invoice close, optionally send a short receipt via gmail.send. Read path is required; send path is demo polish.

6. Killer Demo Script (H) — 3 Minutes

Rehearse this sequence twice before presenting. Do not ad-lib extra features live. The scorecard is the closing artifact.

Time	Beat	What You Do / Show
0:00–0:20	Pitch	"Your books lie until payments, emails, and client names agree. Ledger Twin is the agent that reconciles them — and proves it with tests."
0:20–0:50	Setup	Show Airtable: client "Jose Martinez Studio", open invoice \$1,000 unpaid. Show empty scorecard baseline.
0:50–1:30	Beat 1 — Wrong name	Fire Stripe payment \$1,000 under payer name "J. Martinez". Triad UI: Same entity? uncertain · Who acts? human. Slack Approve card appears.
1:30–2:00	Beat 2 — Approve	Click Approve in Slack. Invoice closes in Airtable. EventLog shows human decision. Scorecard updates.
2:00–2:30	Beat 3 — Duplicate	Re-fire identical Stripe webhook. Idempotency Guard blocks. blocked_duplicates +1. No second receipt. Triad: Already happened? YES.
2:30–3:00	Close	Open scorecard: duplicate-catch rate, eval pass count from `/eval/run`. Point at LangGraph diagram. Stop talking.

Backup beat (if time): Show Gmail email "paid via e-transfer \$1000" for the same client → matched without creating a duplicate payment because idempotency/amount+day hash aligns with prior Stripe event or is clearly linked as secondary evidence.

7. One-Day Build Plan

Hours	Focus	Done Means
0–1	Accounts & Airtable	Stripe test products/invoices; Airtable base (4 tables); Slack app; Gmail OAuth; ngrok/Railway URL.
1–2	Webhook spine	FastAPI Stripe receiver; verify signature; Normalize → EventLog raw write.
2–3	Idempotency Guard	Hash + SQLite UNIQUE; 3 unit tests; blocked_duplicates path.
3–5	Entity Resolver	RapidFuzz thresholds + Claude middle-band prompt; wire Airtable Clients.
5–6	Match + Strict Policy	Payment↔invoice logic; enforce G rules; partial → Slack.
6–7	Slack HITL	Block Kit + triad fields; interactivity callback; pending action execute/discard.
7–8	Scorecard + Triad UI	`GET /scorecard`; dashboard feed; event cards with three questions.
8–9	Eval harness	`POST /eval/run` + fixtures for suite below; fix failures.
9–10	Demo rehearsal	Run killer script twice; freeze scope; prepare LangGraph screenshot.

8. Verification Test Suite

Run via `POST /eval/run` before demo. Every feature above has at least one dedicated test. Pass criteria are binary — fix before going on stage.

ID	Test	Covers	Setup / Inject	Expected Behavior	Pass Criteria
T1	Clean single payment	F2,F3,F6	One Stripe payment_int ent.succeeded for exact-match client + exact amount	Auto-match, close invoice, optional 1 receipt, 0 Slack	Invoice=paid; 1 EventLog; 0 escalations
T2	Duplicate webhook	F1,F5,F6	Identical Stripe webhook payload twice within 5s	First processed; second blocked; triad Already happened=yes	1 receipt max; 1 blocked_duplicates row
T3	Fuzzy name (middle band)	F2,F4,F5, G	Payer "J. Martinez"; client "Jose Martinez Studio"; exact \$	Claude judgment OR Slack; never silent duplicate client	Correct merge or human escalate; no false new client
T4	No match at all	F2,F4,G	Payment name/email resembles no client	New client needs_review=true + Slack	No false merge; Slack sent
T5	Partial / near-miss amount	F3,F4,G	Payment short by fixed \$ vs open invoice	Not fully paid; delta shown; Slack alert	Status≠paid; remaining correct
T6	Strict policy block	G,F4	Name score ≥90 but amount off by \$1+	No auto-close; Slack only	Invoice still open until human
T7	Out-of-order webhook	F1	Webhook timestamp earlier than one already processed	Use event timestamp; process if new external_id; no crash	Correct ordering in EventLog; no false collision on different ids
T8	Gmail payment signal	F8,F2	Email: "paid via e-transfer \$1000" for known client	Normalized Event; match or evidence link; no crash on parse miss	EventLog entry; client resolved or escalated
T9	Slack Approve path	F4,F5	Low-confidence pending action → click Approve	Mutation executes; human_decision=approve logged	Invoice/client state updated; scorecard reflects
T10	Slack Reject path	F4	Pending action → click Reject	No Airtable mutation; decision logged	State unchanged; EventLog has reject
T11	Ambiguity triad present	D,F5	Any processed event (auto or HITL)	Event card / API includes three answers + confidence	`GET /events/{id}` returns triad fields
T12	Scorecard aggregates	B,F6	After T1–T11 fixtures	`GET /scorecard` returns rates + counts	duplicate_catch_rate, escalation_rate, eval_pass_count present
T13	Eval harness smoke	E,F7	`POST /eval/run` with fixture pack	All selected cases execute; results persisted	HTTP 200; scorecard.eval_results length ≥ 8
T14	Malformed Stripe payload	F1 spine	Webhook missing external_id or amount	Log error; skip; no crash; no corrupt Airtable row	Error visible in EventLog; app still healthy
T15	Idempotent receipt	F1,F8	Force re-process attempt of already-closed payment	Second receipt not sent	gmail send count stays 1

9. Minimum API Surface

Method	Path	Purpose
POST	/webhooks/stripe	Stripe events; verify signature; enqueue graph
POST	/webhooks/slack/interact	Block Kit button payloads; resume pending action
POST	/triggers/gmail-poll	Manual/cron poll of payment-related messages
GET	/scorecard	Aggregated reliability metrics for dashboard
GET	/events	Recent EventLog + triad annotations
GET	/events/{id}	Single event with triad + confidence + outcome
POST	/eval/run	Inject fixture suite; return pass/fail matrix
GET	/health	Liveness for Railway/ngrok checks

9.1 Scorecard Payload (Shape)

```
{
  "entity_resolution_accuracy": 0.0-1.0,
  "duplicate_catch_rate": 0.0-1.0,
  "false_merge_rate": 0.0-1.0,
  "escalation_rate": 0.0-1.0,
  "silent_wrong_completions": 0,
  "eval_pass_count": 0,
  "eval_total": 0,
  "recent_events": [ ... ]
}
```

10. Definition of Done & Winning Checklist

Ship bar (must have before demo)

- All 4 services connected in test/sandbox mode with real round-trips
- LangGraph path handles Stripe → resolve → match → route → execute/HITL
- Idempotency Guard blocks live duplicate webhook on stage
- Ambiguity triad visible on event cards and Slack messages
- Scorecard shows live numbers after `/eval/run`
- Killer demo script rehearsed twice end-to-end
- T1, T2, T3, T5, T9, T12, T13 passing hard

Nice-to-have (only if ahead by Hour 7)

- Gmail receipt send on auto-close
- Chart.js sparklines on scorecard
- LangGraph visualization screenshot in slides

Explicitly out of scope today

- Notion dual-write, Celery/Redis, production multi-tenant auth
- Paper Trail contract PDF parsing, Calendly, Circuit Breaker middleware
- Training a custom model — Claude + RapidFuzz only

Bottom line: Build the reliability story, not a kitchen-sink finance app. Win the room with triad + duplicate block + scorecard — in that order.

Generated for build day · 2026-09-13 · Ledger Twin v2 (default package A+B+D+E+G+H)